ML для старта в Data Science

3. Градиентный спуск и его модификации

Жарова Мария

proglib.academy

План занятия, теория + практика

- Классический градиентный спуск
  - вспоминаем теорию
  - схема построения оптимизационных алгоритмов
  - проблемы классической вариации
- Модификации градиентного спуска
  - стохастический градиентный спуск
  - мини-батч
  - AdaGrad
  - RMSProp
  - Adam
- Регуляризация
  - L1, L2 и их комбинация
  - ранняя остановка

Пишите вопросы в чат в любое время! Если писать много – обсудим голосом в специальных перерывах :)

Классический градиентный спуск

Recap: градиент

proglib.academy

Воспользуемся идеей из непосредственно основной характеристики экстремума – *производной.*

Ранее мы говорили, что для функций многих переменных упорядоченный набор частных производных называется **<u>вектором градиента.</u>**

$$z = x^2 + y^2$$

$$\frac{\partial z}{\partial x} = 2x, \quad \frac{\partial z}{\partial y} = 2y$$

$$\nabla z = \left(\frac{\partial z}{\partial x}, \frac{\partial z}{\partial y}\right) = (2x, 2y)$$

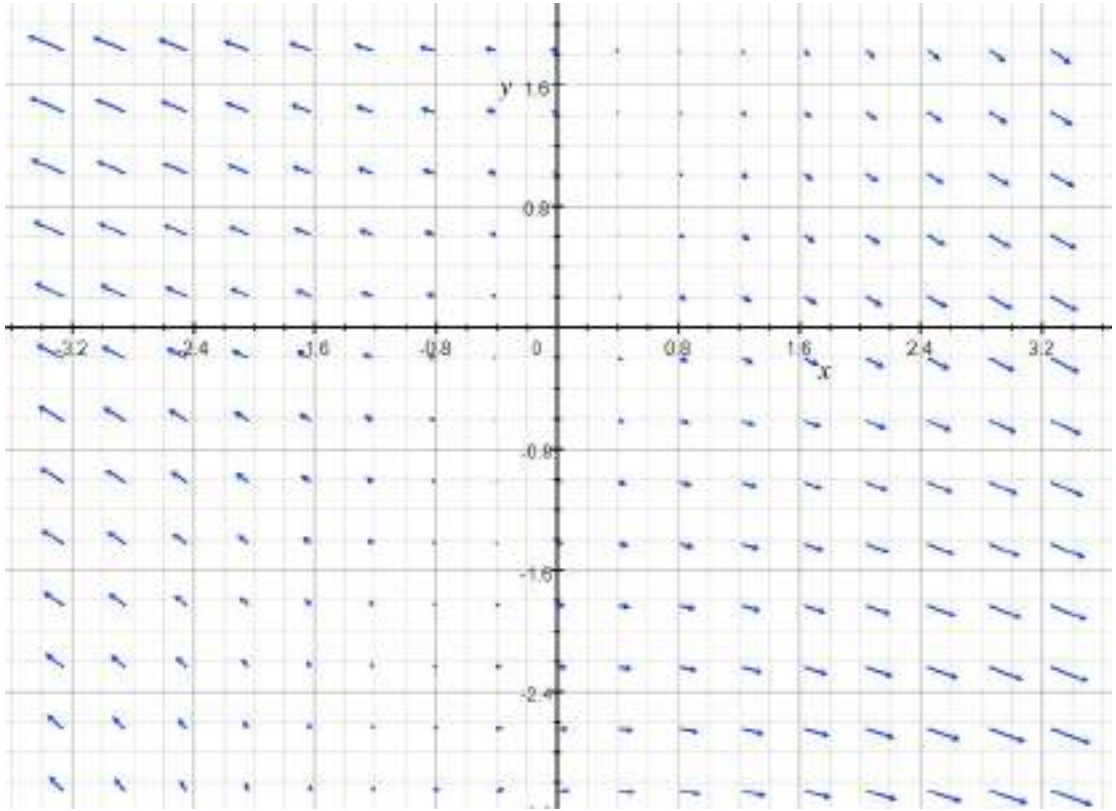
Вспомним, что он значит геометрически…

Recap: градиент и антиградиент

<u>Градиент</u> показывает направление наискорейшего роста функции.

<u>Антиградиент</u> показывает направление наискорейшего убывания функции.

Можно визуализировать все возможные векторы градиента для функции – получим **<u>векторное поле.</u>**



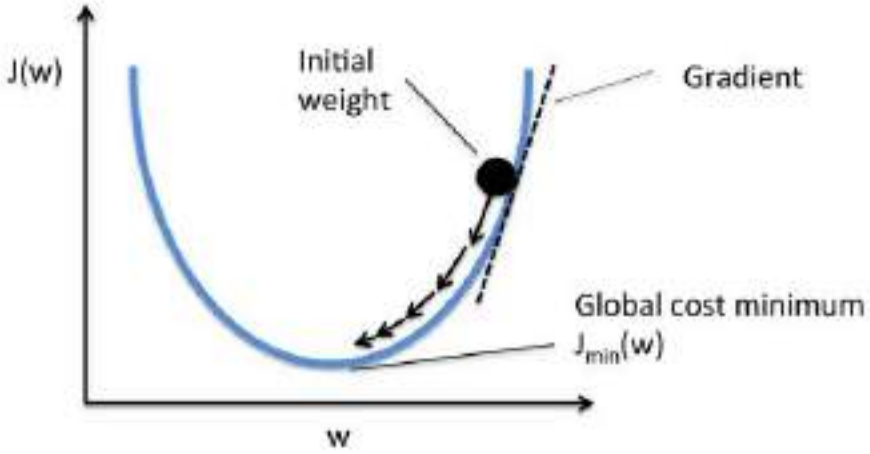
Классический градиентный спуск

Будем двигаться в сторону антиградиента ⇒ придём в минимум функции!

$$w_{k+1} = w_k - \eta \cdot \nabla_w L(w_k)$$

<u>Обозначения:</u>

- η – шаг или learning rate,
- $L(w)$ - функция ошибки/потерь/loss
- $\nabla_w L(w_k)$ – её градиент в точке w_k



Классический градиентный спуск

Будем двигаться в сторону антиградиента ⇒ придём в минимум функции!

$$w_{k+1} = w_k - \eta \cdot \nabla_w L(w_k)$$

<u>Обозначения:</u>

- η – шаг или learning rate,
- $L(w)$ - функция ошибки/потерь/loss
- $\nabla_w L(w_k)$ – её градиент в точке w_k

❌ **Проблема: можно застрять в локальном минимуме + медленно**

Модификации градиентного спуска

proglib.academy

Что можно изменить в формуле?

$$w_{k+1} = w_k - \eta \cdot \nabla_w L(w_k)$$

- η – learning rate ✅
- добавлять новые слагаемые в формулу ✅
- модифицировать набор данных для обновления градиента ✅
- $L(w)$, $\nabla_w L(w_k)$ – функция ошибки и её градиент ❌

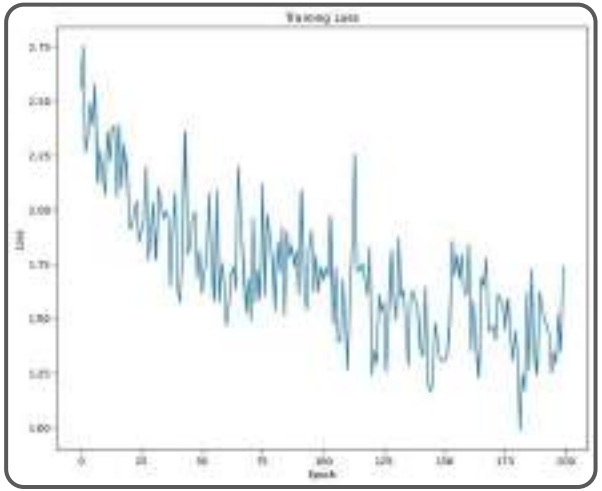
Стохастический градиентный спуск

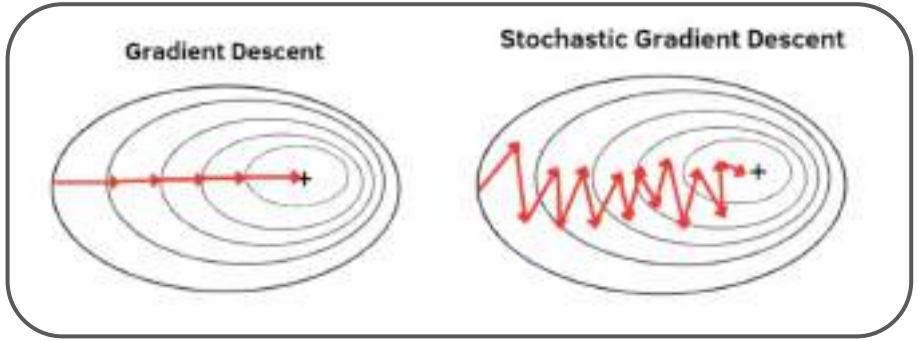
<u>Идея:</u> градиент считается не на всех данных, а на *одном случайном примере*.

$$w_{k+1} = w_k - \eta \cdot \nabla_w L(w_k, x_i)$$

здесь x_i – случайно выбранный объект обучающей выборки.

❌ *Минусы: шумно.*
✅ *Плюсы: быстро, подходит для работы онлайн*
+ можно выбраться из локальных минимумов.





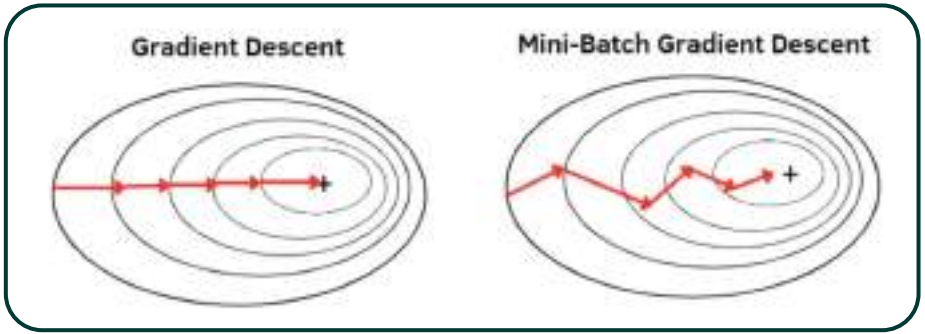
Мини-батчевый градиентный спуск

<u>Идея:</u> градиент считается не на всех данных, а на *мини-батче*.

$$w_{k+1} = w_k - \eta \cdot \frac{1}{m} \sum_{i=1}^{m} \nabla_w L(w_k, x_i)$$

здесь x_i – случайно выбранный объект обучающей выборки, m – размер мини-батча, лучше брать его как 2^n, $n \in N$ (из-за особенностей GPU, CPU и библиотек:))

✅ *Компромисс между точностью и скоростью:*
- быстрее, чем GD,
- стабильнее, чем SDG.



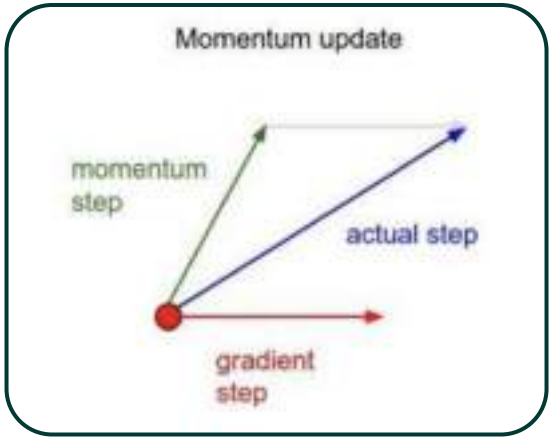
Добавление моментума

<u>Идея:</u> добавляется "инерция", чтобы сглаживать траекторию обновлений.

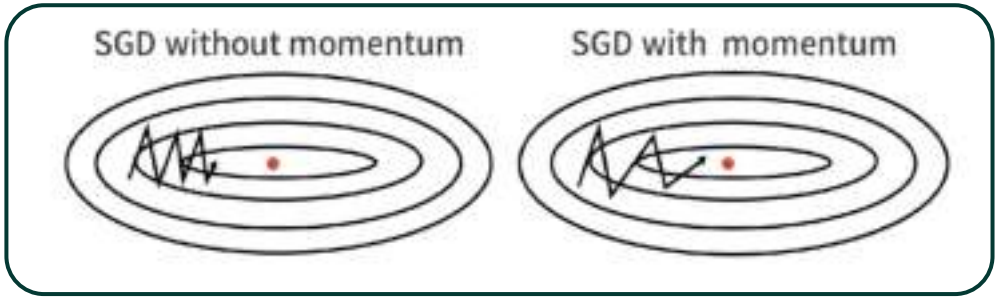
$$v_{t+1} = \beta \cdot v_t + \nabla_w L(w_t)$$

$$w_{t+1} = w_t - \eta \cdot v_{t+1}$$

здесь v_t — "скорость" (скользящее среднее градиентов), $\beta \in [0,1)$ — коэффициент инерции (обычно ~0.9).



✅ *Ускоряет обучение в вытянутых областях функции потерь*
✅ *Сглаживает траекторию обучения*
✅ *Помогает выбраться из седловых точек*



AdaGrad (Adaptive Gradient)

proglib.academy

<u>Идея:</u> рассчитывается индивидуальный learning rate на каждый параметр.

$$G_{t+1} = G_t + (\nabla_w L(w_t))^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_{t+1}} + \epsilon} \cdot \nabla_w L(w_t)$$

здесь G_t — накопленная сумма квадратов градиентов (считается поэлементно), ϵ — маленькое число для стабильности и защиты от деления на 0.

✅ Автоматически адаптирует скорость обучения

✅ Идеален для разреженных данных

❌ G_t только растёт, и lr может стать слишком маленьким ⇒ остановка обучения

RMSProp (Root Mean Square Propagation)

proglib.academy

<u>Идея:</u> вместо накопления всех градиентов храним *скользящее среднее квадрата градиента*, чтобы шаг не обнулялся со временем.

$$E_{t+1} = \rho \cdot E_t + (1-\rho) \cdot (\nabla_w L(w_t))^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{E_{t+1}} + \epsilon} \cdot \nabla_w L(w_t)$$

здесь $\rho \approx 0.9$,
E_t — экспоненциальное скользящее среднее квадрата градиентов.

✅ Исправляет проблему затухания шага AdaGrad

✅ Хорошо работает с нестабильными функциями потерь

Adam (Adaptive Moment Estimation)

proglib.academy

<u>Идея:</u> лучшие идеи Momentum и RMSProp.

$g_t = \nabla_w L(w_t)$ - градиент

$m_{t+1} = \beta_1 \cdot m_t + (1 - \beta_1) \cdot g_t$ – 1-й момент (средний градиент)

$v_{t+1} = \beta_2 \cdot v_t + (1 - \beta_2) \cdot g_t^2$ – 2-й момент (ср. квадрат гр-та)

$\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}$ – коррекция смещения

<u>Итоговая формула:</u>

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}} + \epsilon} \cdot \hat{m}_{t+1}$$

✅ Считает скользящее среднее градиентов (как Momentum)

✅ Считает скользящее среднее квадратов градиентов (как RMSProp)

✅ Добавляет коррекцию смещения на ранних итерациях

Adam (Adaptive Moment Estimation)

Итоговая формула:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}} + \epsilon} \cdot \hat{m}_{t+1}$$

✅ Быстро сходится

✅ Хорошо работает на шумных данных и нестабильных градиентах

✅ Хорошая реализация "из коробки" в PyTorch и TensorFlow

❌ Может "перепрыгивать" хорошее решение и переобучаться из-за сильной адаптивности

❌ Медленнее адаптируется при fine-tuning

Шаблон для оптимизационных алгоритмов

input:

- функция потерь **L(w)**
- начальное приближение $\mathbf{w_0}$ (веса, параметры)
- шаг обучения **η** (learning rate)
- дополнительные параметры (например, **λ** — регуляризация, momentum и т.п.)
- критерий остановки (кол-во итераций или “сходимость”)
- технические переменные: списки для хранения истории обучения, счётчик итераций

while(не достигнут критерий остановки):

- вычислить градиент: **∇L(w)**
- вычислить направление обновления d (например: **d ← -∇L(w)**, учесть регуляризации, моментум и т.п.)
- обновить параметры по рекуррентной формуле вида: **w ← w + η * d**
- (опционально) сохранить текущее значение ошибки или функции потерь, обновить счётчик итераций

output:

- оптимальное значение весов модели w*
- (опционально) история значений функции потерь / история обучения

Ещё проблемы на подумать

С локальными минимумами решили, но это ещё не всё…

<u>Могут быть проблемы с:</u>

- затуханием градиента ⇒ обучение прекращается
- взрывом градиента ⇒ обучение нестабильно или в ответах NaN
- нестабильностью обучения (смотрим по loss)

Пути решения

Проблема	Как решить?
Переобучение	Регуляризация (L1/L2), сбор большего числа данных, специальные методы для нейросетей
Медленная сходимость	Adam/RMSProp, подобрать learning rate, использовать learning rate scheduler
Застревание в локальном минимуме	Моментум или Adam, добавить стохастичность
Затухающие градиенты	Нормализация градиентов, для нейросетей спец. методы
Взрывающиеся градиенты	Gradient clipping, уменьшение learning rate
Нестабильное обучение	Нормализация данных, уменьшение learning rate

Регуляризация

Recap: что такое переобучение

proglib.academy

<u>Переобучение (overfitting)</u> – это ситуация, когда модель слишком хорошо запоминает тренировочные данные, но плохо обобщает их на новые тестовые (зазубривает).

<u>Признаки переобучения:</u>

- Низкая ошибка на обучающих данных, но высокая на тестовых.
- Высокие коэффициенты (веса) модели.

<u>Чем может быть вызвано:</u>

1. *Проблемы в данных:* мультиколлинеарность, шум, выбросы, разный масштаб признаков, утечки данных и их недостаток.
2. *Проблемы на этапе моделирования:* слишком сложная модель или чрезмерно долгое обучение.

1.1. Проблема разного масштаба признаков

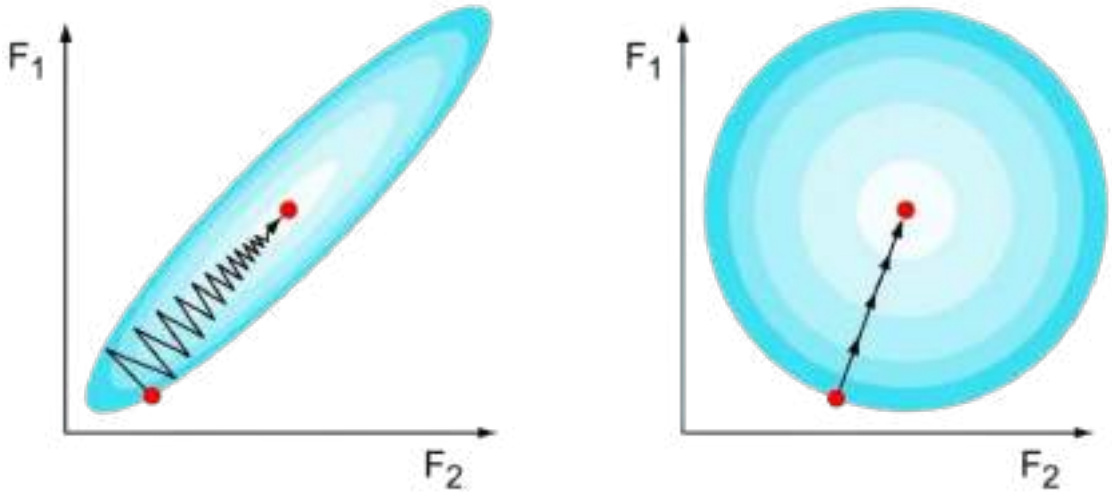
При работе с ML-моделями данные могут содержать признаки с *разными единицами измерения и порядками величин*. Например:

- Возраст: 25, 40, 60 (диапазон десятков)
- Площадь дома: 50, 100, 200 (диапазон сотен)
- Зарплата: 30 000, 60 000, 100 000 (диапазон тысяч)

Это создает проблемы для алгоритмов, особенно использующих метрики расстояния (например, метод ближайших соседей, градиентный спуск).

1.1. Проблема разного масштаба признаков

Наглядно проблема показана на рисунке: слева процесс сходимости градиентного спуска на исходных немасштабированных данных, а справа – на масштабированных.



<u>Как итог:</u> масштабирование улучшает сходимость алгоритмов ⇒ повышает качество. Также это критически важно при регуляризации и поиске расстояний между векторами.

1.2. Переобучение из-за мультиколлинеарности

Когда 2 признака сильно коррелируют, в матрице X их столбцы почти пропорциональны (линейно зависимые), что ведет к нестабильности в решении линейной системы.

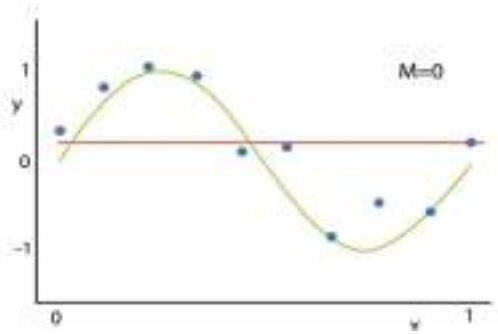
Это приводит к большим коэффициентам регрессии, которые зависят от малейших изменений данных.

Модель начинает запоминать шум вместо выявления общих закономерностей.

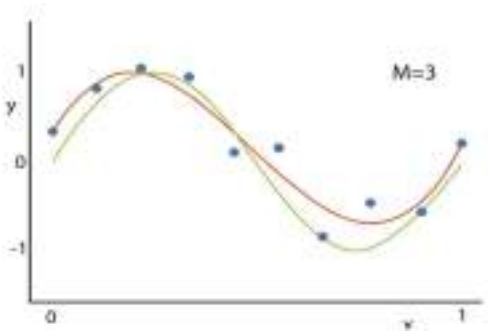
<u>Пример:</u> если у нас 2 сильно коррелированных признака (например, длина и площадь комнаты), модель может назначить им огромные по модулю, но разнонаправленные коэффициенты. Это делает модель нестабильной и увеличивает вероятность переобучения.

2.1. Переобучение из-за “сложной” модели

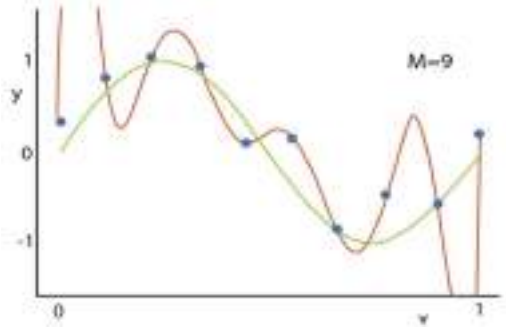
<u>Пример:</u> в качестве модели можно взять *полиномиальную регрессию* – та, которая “пытается аппроксимировать” зависимость не прямой, а полиномом заданной степени. Если переборщить со степенью полинома, может случиться переобучение:



Недообучение



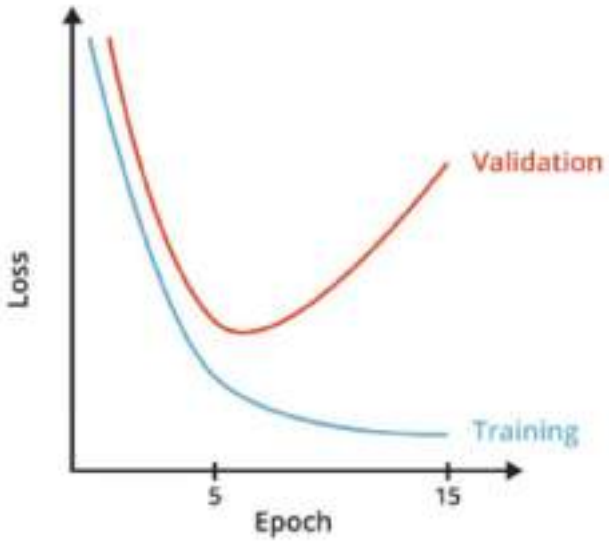
Оптимально!



Переобучение

2.2. Переобучение из-за долгого обучения

<u>Пример:</u> если слишком долго обучать модель на одних и тех же данных, она может начать “запоминать” шум и случайные колебания вместо общей закономерности.
В итоге — переобучение.



Как бороться с проблемами в данных?

proglib.academy

<u>“Первая группа” методов борьбы с переобучением:</u>

- Удалять мультиколлинеарные признаки
- Убирать шум и выбросы (фильтрация, усечение)
- Нормализовать признаки (масштабирование)
- Проверять утечки данных – по времени, по признакам, между train и test
- Добавлять больше данных или использовать аугментации при недостатке
- Удалять нерелевантные/мусорные признаки

Как бороться на уровне модели?

<u>Регуляризация</u> добавляет штраф к функции потерь, заставляя модель тем самым уменьшать масштабы коэффициентов линейной регрессии.

<u>Виды регуляризации:</u>

- L_2, добавляем к loss $\lambda \sum_{j=1}^{n} w_j^2$ (плюс – она всюду дифференцируема, штраф сильнее)
- L_1, добавляем к loss $\lambda \sum_{j=1}^{n} |w_j|$ (минус – недифференцируема, зато обнуляет веса)
- $L_1 + L_2$ (Elastic Net)

```
from sklearn.linear_model import Lasso # L1
from sklearn.linear_model import Ridge # L2
from sklearn.linear_model import ElasticNet # L1+ L2
```

линейная регрессия с разными видами регуляризаций

Как бороться на уровне модели?

Ранняя остановка – тоже способ предотвратить переобучение: обучение прерывается, когда качество на валидации перестаёт улучшаться.

Идея буквально следующая:
Модель учится до тех пор, пока валидационная ошибка уменьшается.
Как только начинает расти — обучение останавливается.

Подходит для любой модели, обучаемой итеративно (в т.ч. градиентным спуском).

Ещё пара лайфхаков

proglib.academy

Сначала поставьте большой learning rate, чтобы модель быстрее сходилась.
→ Но следите, чтобы loss не скакал.

Если модель "застряла", уменьшите learning rate во время обучения.
→ Это поможет найти более точный минимум.

Включите scheduler — это автоматическая регулировка learning rate во времени.
Примеры:

- StepLR — каждые N эпох уменьшает lr в несколько раз.
- ReduceLROnPlateau — уменьшает lr, если метрика не улучшается.
- CosineAnnealingLR — красиво "затухает" lr к нулю.

Спасибо за внимание!